Markov Decision Process

CS3263

Foundations of Artificial Intelligence

Version 5.0

What we have learned so far…

- Classical Planning
  - Sequential, deterministic, fully observable
- Decision Analysis
  - Episodic or sequential, stochastic, fully observable
- Up Next: Decision Making Under Uncertainty and Over Time
  - Sequential, stochastic, fully or partially observable

Topics

- Markov Decision Process
  - Model formulation and solution
  - Bellman Equation and Q-function
- Basic Algorithms for solving MDPs
  - Value iteration
  - Policy iteration

Notes: Sequential Decision Problems

- Sequential Decisions Under Uncertainty
  - Decisions made over time under uncertainty
  - Utility depends on the full action sequence
  - Search and planning are special cases
- Examples:
  - *Markov Decision Process (MDP)*:
    Fully observable, stochastic, with long-term rewards
    Solved efficiently in real-world domains: robotics, logistics, games
  - *Reinforcement Learning (RL):*
    Learns to act in unknown environments through feedback

Example: Persy the Mars Rover Adventures

Percy is exploring Mars:

- Starting at the base station with limited energy.
- Percy can
  - Move to explore,
  - Collect a soil sample,
  - Transmit data back to Earth, or
  - Recharge using solar panels.
- Moving and collecting are uncertain
  - Rocky terrain may drain more energy, solar recharge may fail if it's cloudy
- Goal:
  - Percy's goal is to maximize useful science data sent back before energy runs out.

Question:

- How should Percy decide what to do at each step, given uncertainty and long-term goals?

Example: Ride-Hailing Fleet Management

Real-world scenario

- A ride-hailing company (e.g., Grab, Uber, Lyft) manages thousands of drivers across a city:
- Each driver must decide whether to
  - Wait for a nearby passenger,
  - Relocate to a busier area, or
  - Refuel/Recharge.
- Passenger demand is uncertain, and traffic/weather affect travel times.
- Goal:
  - To maximize long-term efficiency: serve more rides, reduce idle driving, and keep drivers profitable.

Question:

- How should the system assign and guide drivers to balance supply, demand, and future opportunities?

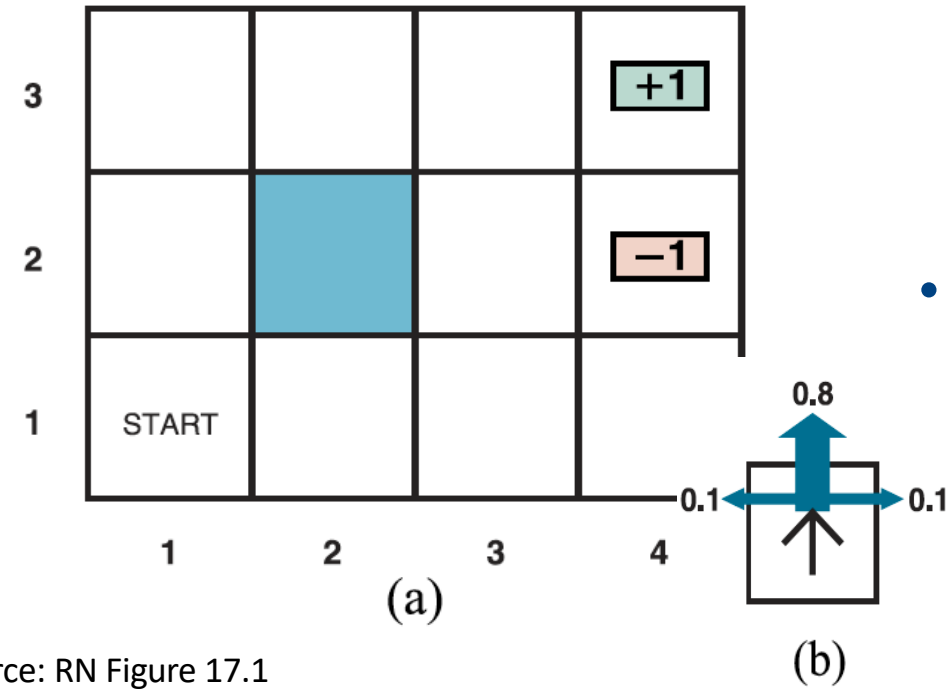
Solving Sequential Decision Problems

- Decision Problem or Model
  - Representation of states, actions, uncertain effects, and preferences (via costs, rewards, or values), time horizon
- Decision Algorithm
  - Input: a decision problem (explicit model or interactive environment)
  - Output: an optimal or near-optimal solution over the time horizon
- Decision Solution
  - An optimal action sequence from an initial state, or
  - An optimal policy that specifies the best action in each state or observation (with respect to costs, values, or preferences)
  - (Optional) A goal state satisfying desired properties in goal-based tasks

Problem Formulation

Define the problem elements

Example: Navigation in Grid World



Source: RN Figure 17.1

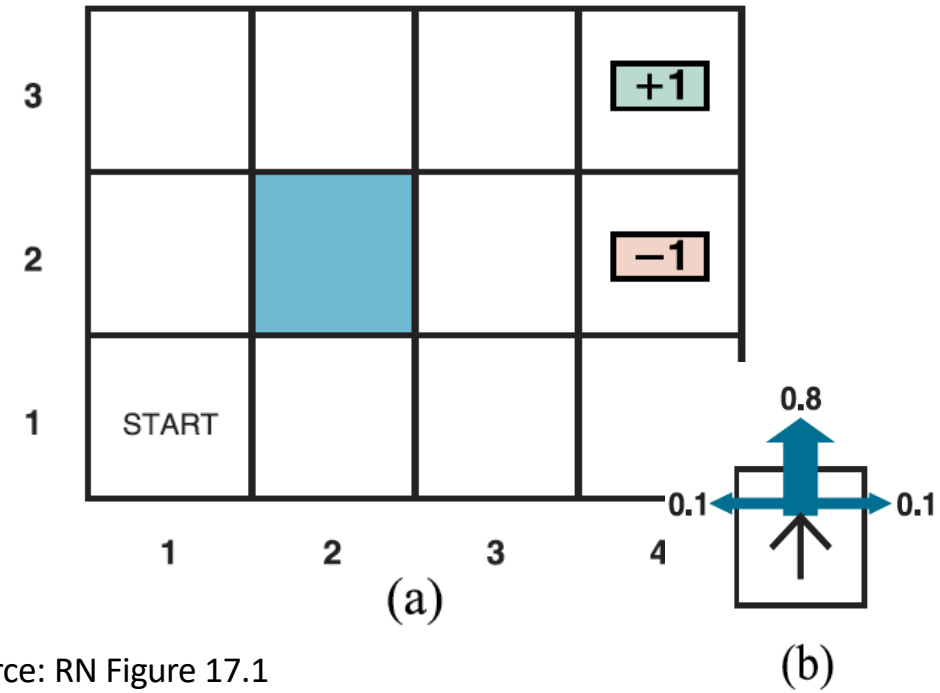
(vs 3 x 4 matrix)

- Fully observable 4 x 3 environment
  - Begin in START state s_0
  - One action a per time step.
  - Terminate when reaching goal states s_g

States marked +1 and -1

- Uncertain action effects:
  - Example: Up, Down, Left, Right:
  - 0.8 - correct direction
  - 0.1 - perpendicular to the left
  - 0.1 - perpendicular to the right

Example: Navigation



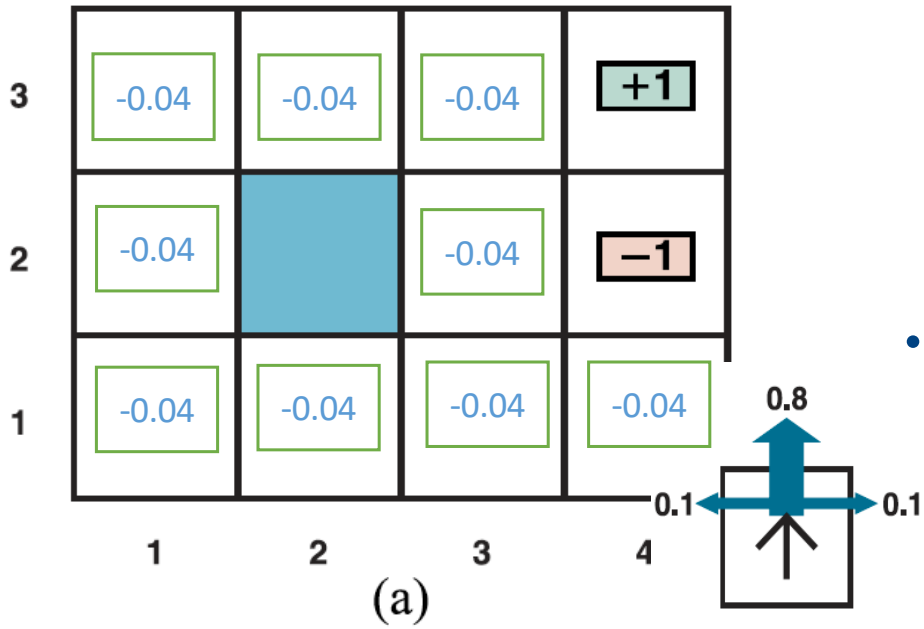
Source: RN Figure 17.1

- Transition model:
  - Define (stochastic) outcome of each action a
  - $P(s'|s,a)$ or $T(s,a,s')$: probability of reaching state s' if action a is done in state s

Markovian assumption:

- Probability depends only on state s and not history of earlier states.

Example: Navigation



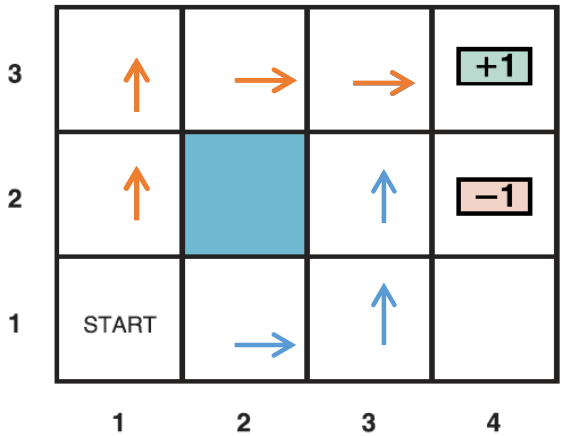
Source: RN Figure 17.1

- Reward model:
  - $R(s, a, s')$ – For transition from s to s' via a; AND/OR
  - $R(s, a)$ – For transition from s via a; AND/OR
  - $R(s)$ – For any transition from/into s
  - Positive or negative rewards, bound by $\pm R_{max}$
- Example:
  - $R(s)$ is -0.04 for all states, except $+1$ or -1 for terminal states

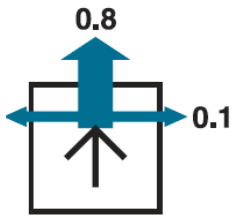
Vs. Utility function or utility of state:

- $U(s)$ – defined on the sequence of states and actions from state s – *environment history* – sum of rewards of the states in the sequence

Example: Navigation



(a)



(b)

Uncertain action effects:

Up, Down, Left, Right:

0.8 - correct direction

0.1 - perpendicular to the left

0.1 - perpendicular to the right

- If action effects are deterministic:
  - What is the optimal policy or action sequence?
- But agent's actions is stochastic
  - What is the chance of getting to the goal with:[U, U, R, R, R]?
  - How to derive optimal policy from the transition function directly?

Markov Decision Process (MDP)

- Formally:
  - An MDP $M \triangleq (S, A, T, R, \gamma)$ consists of:
  - A set S of states
  - A set A of actions
  - A transition function $T: S \times A \times S \rightarrow [0,1]$ that satisfies the Markov property such that:

$$\forall s \in S, \forall a \in A: \sum_{s' \in S} T(s, a, s') = \sum_{s' \in S} P(s'|s, a) = 1$$

  - A reward function $R: S \rightarrow \Re$ or $R: S \times A \rightarrow \Re$ or $R: S \times A \times S \rightarrow \Re$
  - A discount factor $0 \leq \gamma \leq 1$
  - Solution is a policy – a function to recommend an action in each state: $\pi: S \rightarrow A$
    - Solution involves careful balancing of risk and reward

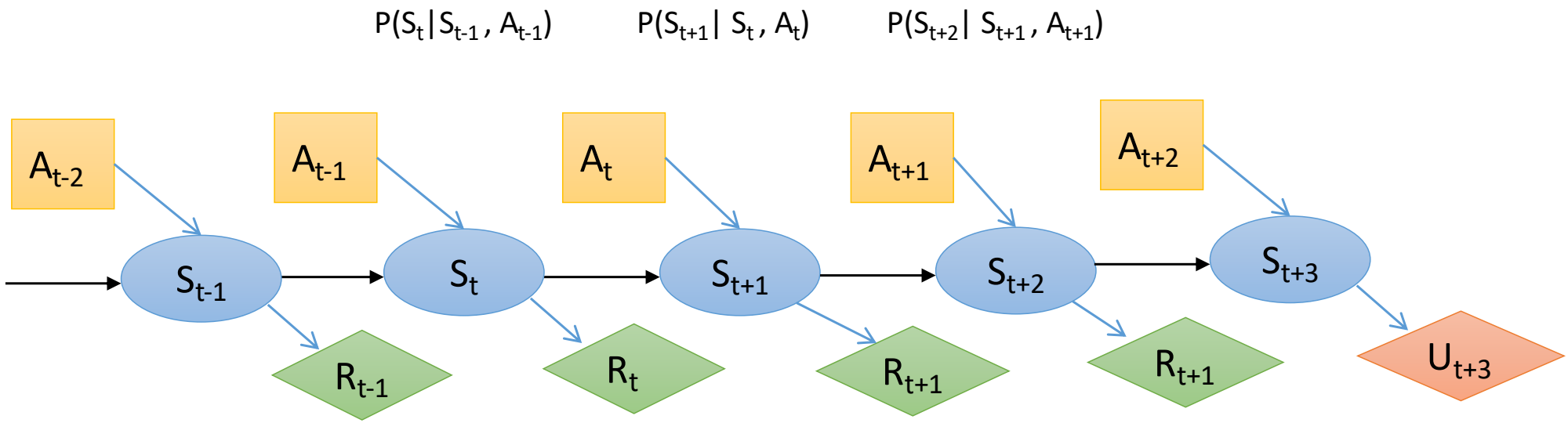
Markov Property

- Definition:
  - Next state depends only on the current state and action — not on full history:

$$P(s_{t+1}|s_t, a_t,\ s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1}|s_t, a_t)$$

- Is this realistic in practice?
  - It's a simplifying assumption
  - Real-world processes may violate it (e.g., with hidden variables or memory effects)
- Why still use it?
  - It reduces complexity of algorithms
  - Many problems can still be well-approximated as Markov processes

MDP as Dynamic Decision Network



What is S_t? A_t? R_t? U_t?

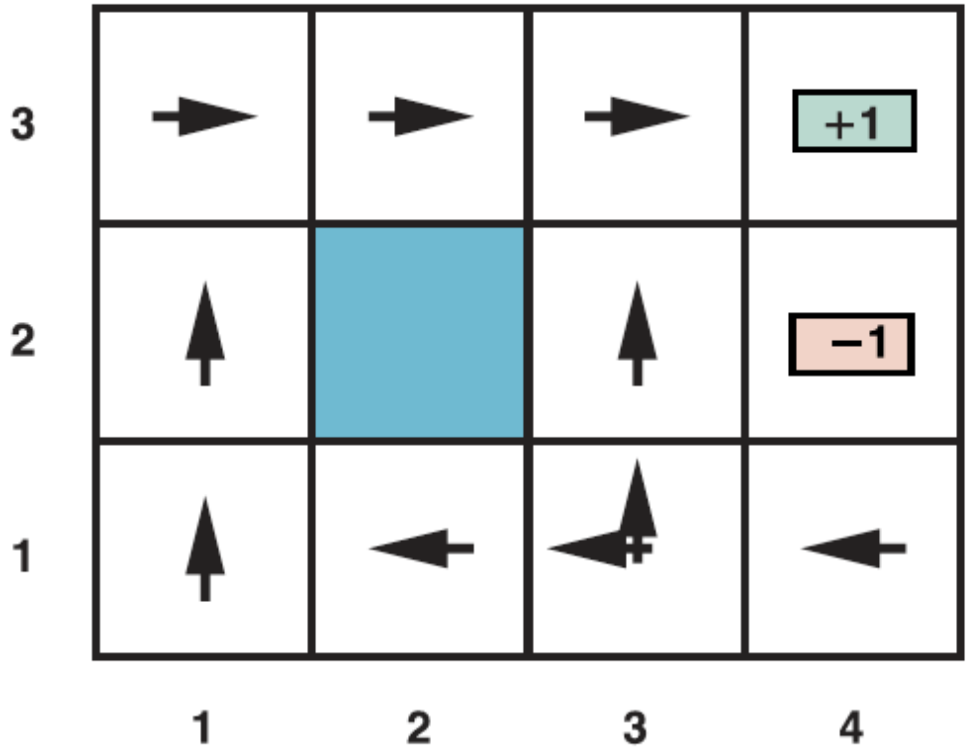
Solution Techniques

Deriving policies – actions to take at each state

Solving MDPs

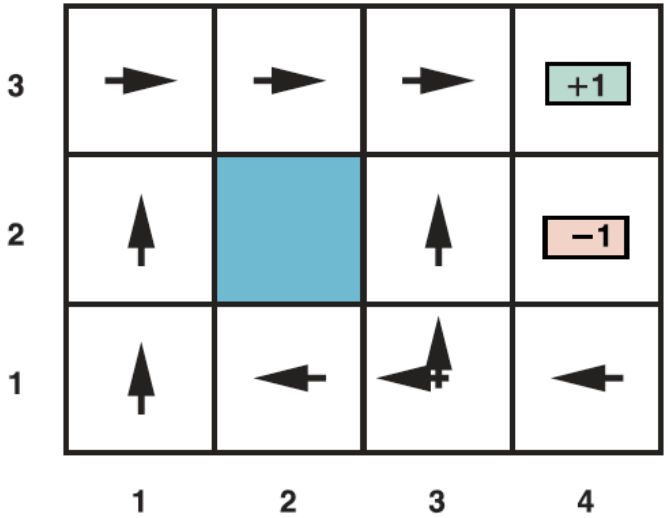
- What does a solution look like?
  - A policy $\boxed{\pi: S \to A}$ maps each state s to an action a.
  - The *quality* of a policy is measured by its expected utility over the state sequences it generates.
  - The optimal policy π^* maximizes expected utility. (MEU Principle)
- An MDP agent uses π^*:
  - It observes the current state s
  - Executes the action $a^* = \pi^*(s)$
  - The policy defines how the agent behaves.

Example: Illustration of π^*

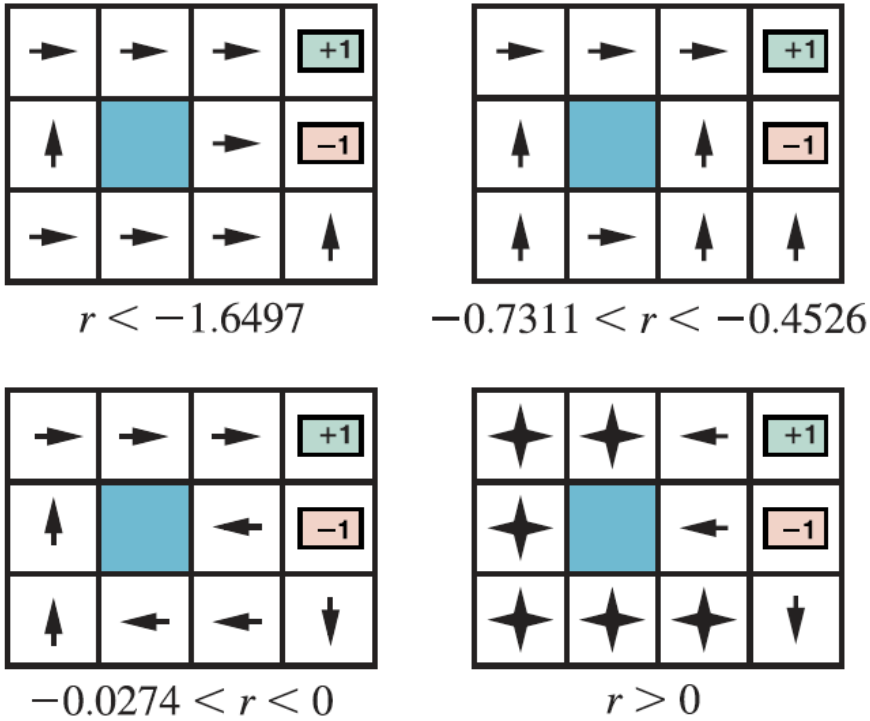


Source: RN Figure 17.2

Example: Balancing Risk and Reward



$r = -0.04$



Changes depending on the value of $r = R(s, a, s')$ for transitions between nonterminal states
There may be many optimal policies for various ranges of r.

Source: RN Figure 17.2

Utility of a State Sequence

- Definition (Utility of a state sequence/trajectory)

  For a state sequence: $\langle s_0, s_1, s_2, \dots \rangle$

  the *utility* is the additive sum of rewards along the sequence

- Finite horizon (N steps):

  $U(\langle s_0, \dots, s_N \rangle) = \sum_{t=0}^{N-1} R(s_t, a_t, s_{t+1}) = R(s_0, a_0, s_1) + R(s_1, a_1, s_2) + \cdots + R(s_{N-1}, a_{N-1}, s_N)$

- Infinite horizon (discounted):

$$U(\langle s_0, s_1, \dots \rangle) = \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) = R(s_0, a_0, s_1) + \gamma R(s_1, a_1, s_2) + \cdots, 0 \leq \gamma < 1$$

- Note:
  - This quantity is often called the *return* and denoted G_t in reinforcement learning (RL)

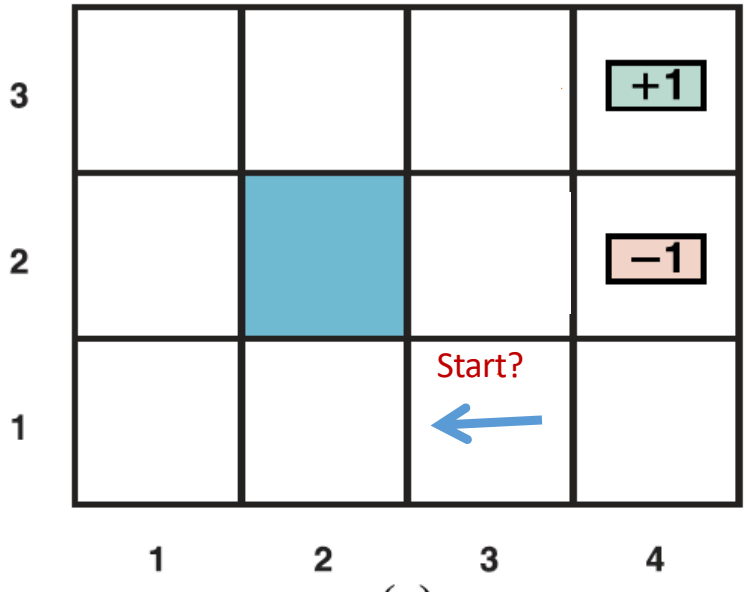
Finite and Infinite Horizon Problems

- Recall:
  - A solution to an MDP is an optimal policy π^*: a mapping from states to actions that maximizes expected utility.
  - The quality of a policy is measured by its expected utility over the state sequences it generates.
- Finite horizon (N steps) $U(\langle s_0, \dots, s_N \rangle) = R(s_0, a_0, s_1) + R(s_1, a_1, s_2) + \cdots + R(s_{N-1}, a_{N-1}, s_N)$
  - Utility of state sequences = sum of rewards up to time N
  - Optimal action may depend on time step → optimal policy can be non-stationary (π_t^*)
- Infinite horizon (discounted) $U(\langle s_0, s_1, \dots \rangle) = R(s_0, a_0, s_1) + \gamma R(s_1, a_1, s_2) + \cdots, 0 \leq \gamma < 1$
  - Utility of state sequences = discounted sum of rewards
  - No fixed deadline
  - Optimal policies are typically stationary (π^*)

Example: Finite Horizon Problem

N = 3?

N = 100?



Source: RN Figurer 17.2

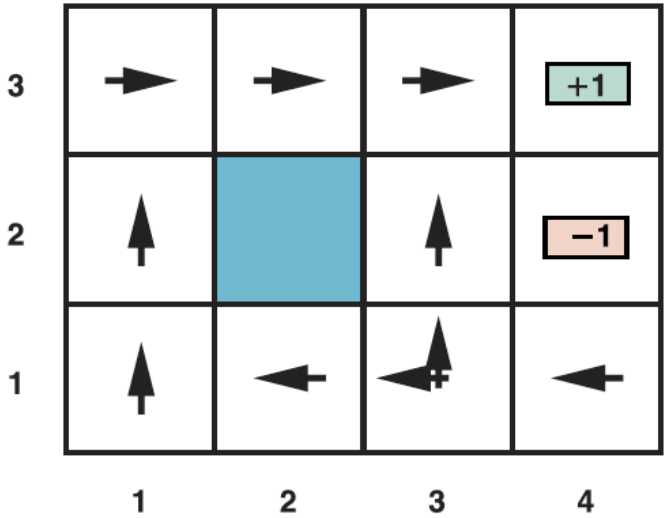
Achieving Finite Rewards in Infinite Horizon Problems

1. Discounted rewards
   - If $\gamma < 1$ and rewards are bounded by $\pm R_{\max}$, then for any infinite sequence:

$$U(\langle s_0, s_1, \dots \rangle) \leq \frac{R_{\max}}{1-\gamma}$$

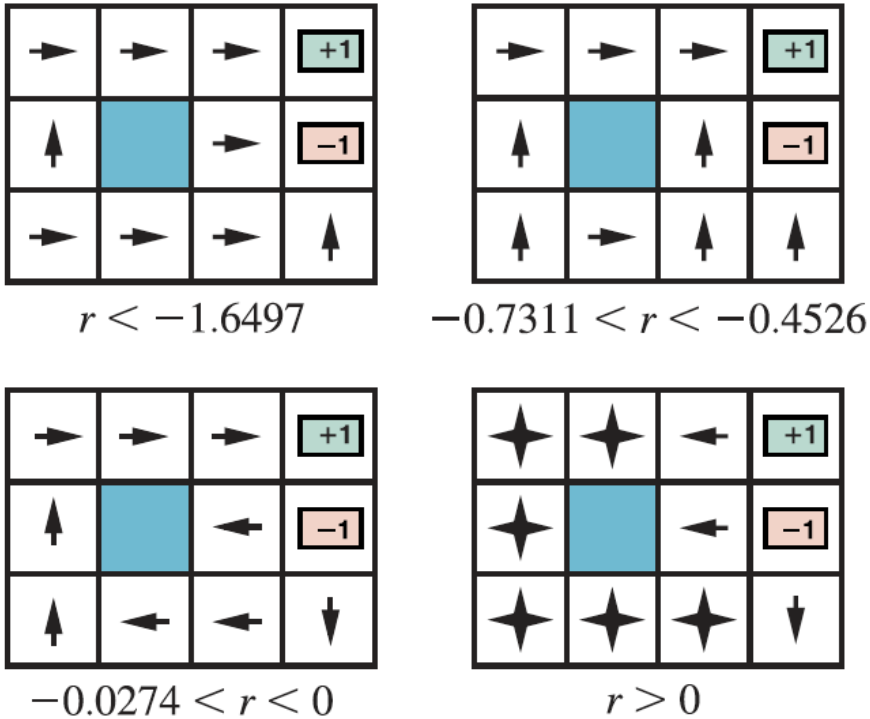
2. Terminal states
   - If the environment has *terminal states* and a *proper policy* (agent always reaches a terminal state), then additive rewards with $\gamma = 1$ are valid.
3. Average reward per time step
   - Evaluate policies by long-run average reward.
   - More difficult to compute and analyze.

Example: Proper and Improper Policies



$r = -0.04$

(a)



(b)

Source: RN Figure 17.2

State Utility Under A Policy

- Policy evaluation: Utility of a state s under policy π:
  - Expected sum of discounted rewards by starting in state s and following π

$$U^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^{t} R(S_t, A_t, S_{t+1}) | S_0 = s\right]$$

- Bellman Expectation Equation:

$$U^{\pi}(s) = \sum_{s'} P\left(s' | s, \pi(s)\right)\left[R(s, \pi(s), s') + \gamma U^{\pi}(s')\right]$$

- Interpretation:
  - Utility of a state under policy π is the expected immediate reward plus discounted utility of successor states, following π.

State Utility and Optimal Policy

- Utility of a state s (Value of a state s):
  - Best achievable expected utility starting from s:

$$U(s) = U^{\pi^*}(s) = \max_{\pi} U^{\pi}(s) = V(s)$$

- Bellman Optimality Equation:

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$$

- Optimal policy (Maximum Expected Utility Principle):

$$\pi^*(s) = \text{argmax}_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$$

- Interpretation:
  - Utility of a state is the maximum expected immediate reward plus discounted utility of successor states.

Bellman Optimality Equation: Infinite Horizon

- Bellman Optimality Equation (from Utility of state):

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$$

- Interpretation:
  - Utility of a state is the maximum expected immediate reward plus discounted utility of successor states
  - Encodes the Maximum Expected Utility (MEU) principle for sequential decisions.
- Notes:
  - One Bellman equation per state → $|S|$equations with $|S|$ unknowns
  - Unique solution gives utilities of all states
  - Basis of value iteration and related algorithms

Q-Function

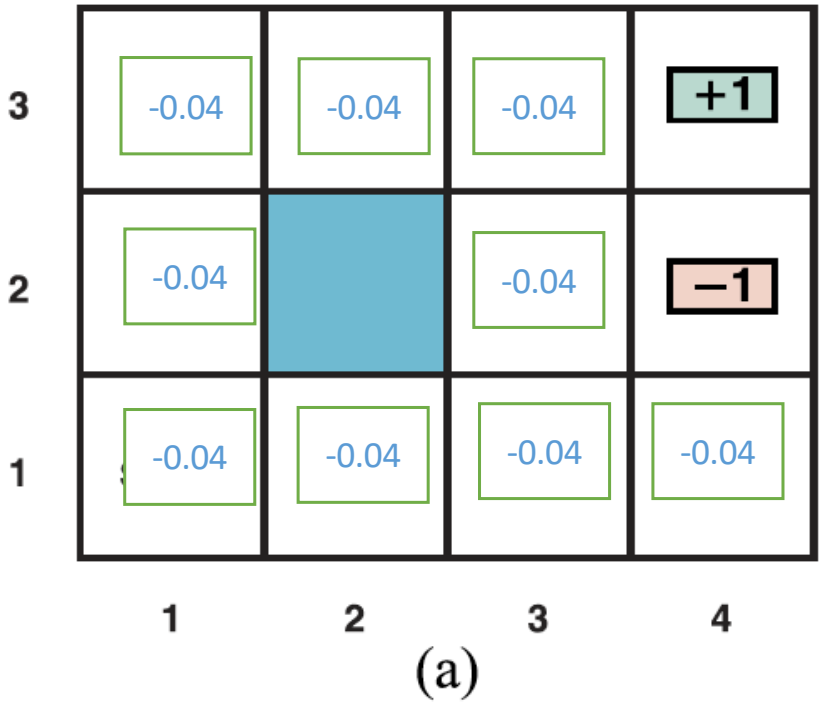
- Q-Function (Action-Utility Function)
  - State utility $U(s)$ is defined by the Bellman Optimality Equation.
  - To select actions, it is useful to evaluate state–action pairs directly.
- Definition (Bellman Optimality Equation for Q):

$$Q(s,a) = \boxed{\sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]}$$

  - State utility: $U(s) = \max_a Q(s,a)$
  - Optimal policy: $\pi^*(s) = \text{argmax}_a Q(s,a)$
- Interpretation:
  - $Q(s,a) =$ expected utility of taking action a in state s, then following the optimal policy.

Example: Utilities of States

$R(s) = -0.04$



(a)

Numbers are $U(s)$

How to compute the $U(s)$ numbers?

Source: RN Figure 17.3

Value Iteration

Solution Method

Value Iteration

- Iterative, repeated Bellman updates of state utilities:
  1. Initialize utilities arbitrarily (e.g., $U(s) = 0$)
  2. Update utilities of all states using:

Bellman update

$$U_{i+1}(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U_i(s')]$$

  3. Repeat until values converge (changes < threshold ϵ)
  4. Extract optimal policy from final utilities:

$$\pi^*(s) = \text{argmax}_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$$

- Key points:
  - Convergence: Approximates utilities until convergence to the unique Bellman solution.
  - Optimality: Extracted policy is guaranteed to be optimal.
  - Complexity: Each iteration is $O(|S|^2|A|)$; total iterations depends on ϵ and γ.

Example: Solving Bellman Equations

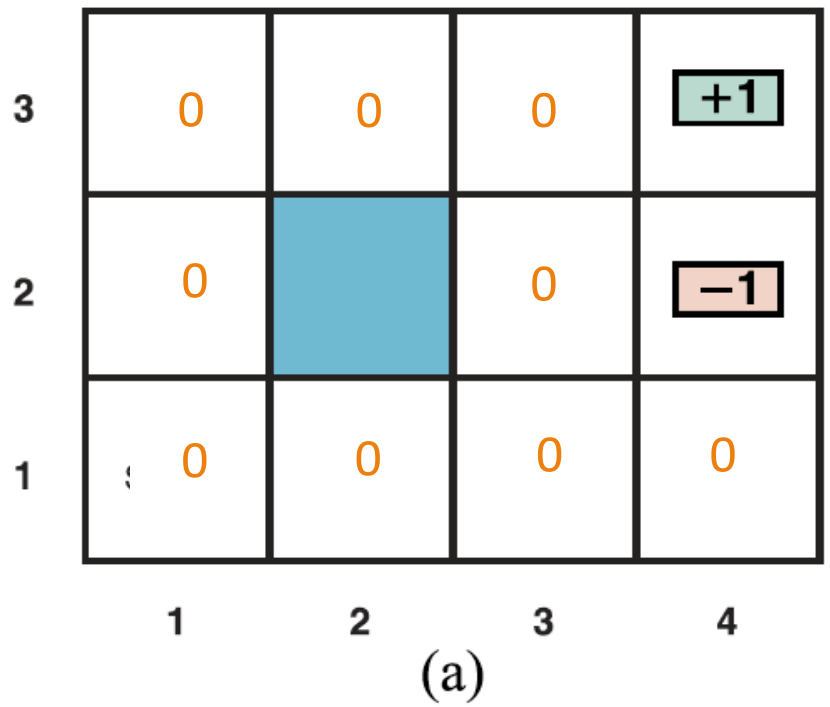
Numbers are $U(s)$

Initialize: $U(s) = 0$

How to compute the $U(s)$ numbers?

Bellman update

$$U_{i+1}(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U_i(s')]$$



(a)

Source: RN Figure

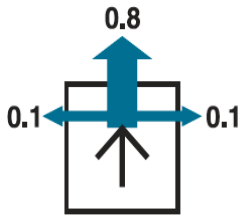
Example: Solving Bellman Equations

$R(s) = -0.04$

Numbers are $U(s)$

How to compute the numbers?
What is the optimal policy?

3	0.8516	0.9078	0.9578	+1
2	0.8016		0.7003	−1
1	0.7453	0.6953	0.6514	0.4279
	1	2	3	4



(b)

For U(1,1): $U_{i+1}(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U_i(s')]$

$$\max\{\ [0.8(-0.04+\gamma U(1,2))+0.1(-0.04+\gamma U(2,1))+0.1(-0.04+\gamma U(1,1))], \quad \text{(U)}$$
$$[0.9(-0.04+\gamma U(1,1))+0.1(-0.04+\gamma U(1,2))], \quad \text{(L)}$$
$$[0.9(-0.04+\gamma U(1,1))+0.1(-0.04+\gamma U(2,1))], \quad \text{(D)}$$
$$[0.8(-0.04+\gamma U(2,1))+0.1(-0.04+\gamma U(1,2))+0.1(-0.04+\gamma U(1,1))]\} \quad \text{(R)}$$

$\pi^*((1,1)) =$

Source: RN Figure 17.3

Demos: Value Iteration

IIIT Hyderabad Virtual Labs

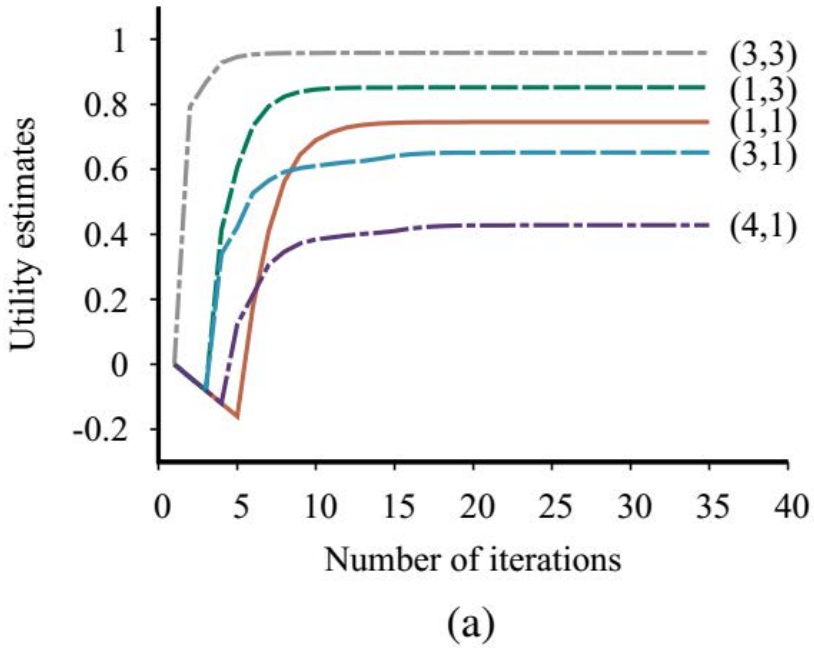
0.000 ←	0.000 →	1.000
0.000	0.000 ←	-1.000
0.000 ←	0.000 ←	0.000 ←

https://ai1-iiith.vlabs.ac.in/exp/value-iteration/simulation.html

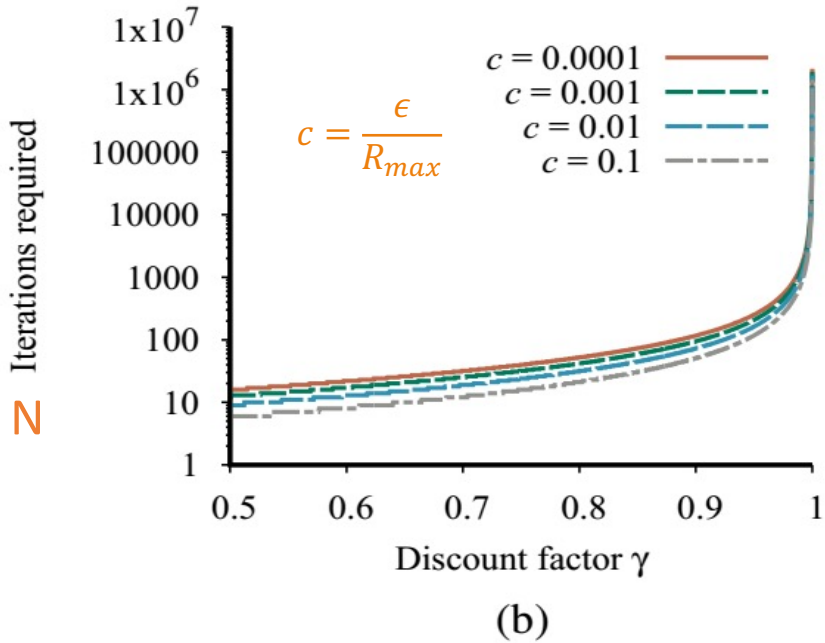
Value Iteration: Why Does It Work?

- Convergence
  - Bellman updates converge to a fixed point: the solution to the Bellman optimality equations.
  - Utilities stabilize once future rewards are fully incorporated.
- Optimal Policy
  - Optimal policy π^* is extracted from converged utilities.
  - Often, a near-optimal policy emerges before full utility convergence.

Notes: Example: Convergence



Evolution of the utilities of selected states using value iteration.

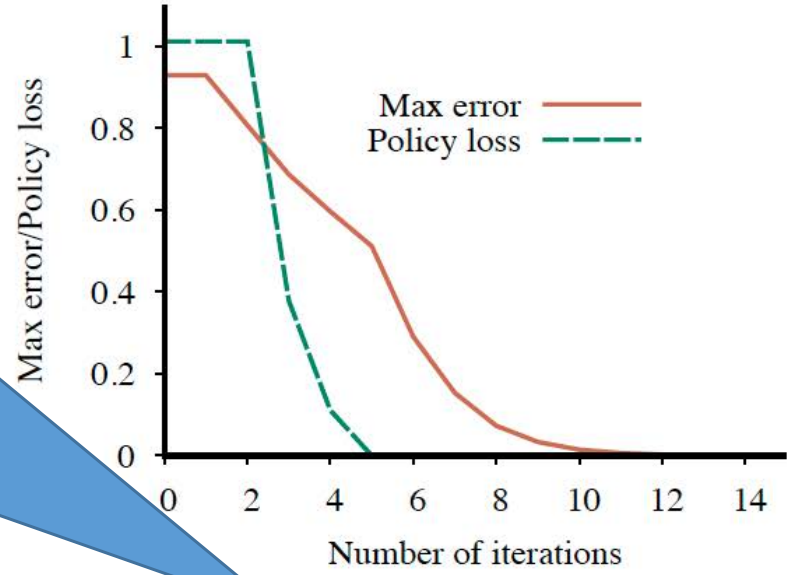


No. of value iterations required to guarantee an error of at most $\epsilon = c \cdot R_{max}$, for different values of c, as function of γ.

Source: RN Figure 17.7

Notes:
Example: Computing Optimal Policy

Max Norm:
Largest difference between any two corresponding elements in two vectors.
It measures the "size" of a vector by the biggest value in it.



$U^{\pi_i}(s)$is utility obtained if π_i is executed starting in state s

Policy loss $\|U^{\pi_i} - U\|$ is the max loss by following π_i instead of π^*

Figure 17.8 The maximum error $\|U_i - U\|$ of the utility estimates and the policy loss $\|U^{\pi_i} - U\|$, as a function of the number of iterations of value iteration on the 4×3 world.

Source: RN Figure 17.8

Policy Iteration

Solution Method

Policy Iteration

- Iterative Steps:
  1. Initialize an arbitrary policy π_0.
  2. Policy Evaluation: Solve for utilities $U^{\pi}(s)$ under π_i :

     Bellman expectation equation

     $$U^{\pi_i}(s) = U_i(s) = \sum_{s'} P(s'|s, \pi_i(s))[R(s,\ \pi_i(s), s') + \gamma U_i(s')]$$
     - For large state spaces, use iterative approximation:

     $$U_{i+1}(s) \ \leftarrow \ \sum_{s'} P(s'|s, \pi_i(s))[R(s,\ \pi_i(s), s') + \gamma U_i(s')]$$
  3. Policy Improvement: Update policy π_{i+1} based on current utilities:

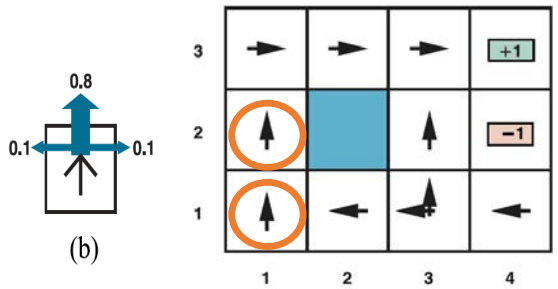
     $$\pi_{i+1}(s) = \arg\max_{a} \sum_{s'} P\,(s'|s, a)[R(s, a, s') + \gamma U_i(s')]$$
  4. Repeat until the policy stabilizes (no change between iterations).
- Key Points:
  - Convergence: Terminates in a finite number of iterations.
  - Optimality: Final policy is guaranteed to be optimal.
  - Complexity: Policy evaluation solves $|S|$ linear equations ($O(n^3)$ for $|S| = n$); iterative evaluation used for large state spaces

Example: Implementing Policy Evaluation

NO max operator

- Policy evaluation (Bellman expectation equation):

$$U_i(s) = \sum_{s'} P(s'|s,\pi_i(s))[R(s,\pi_i(s),s') + \gamma U_i(s')]$$
$$= R(s) + \gamma \sum_{s'} P(s'|s,\pi_i(s))U_i(s')$$



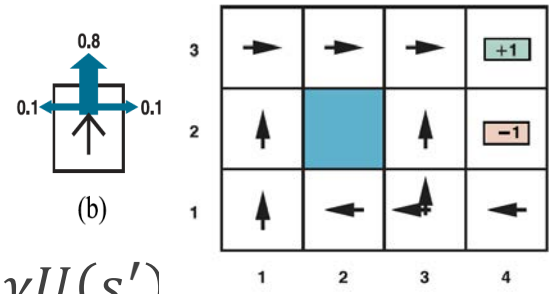
Assume: $\pi_i(s)$

- Start with a "fixed" policy:

$$U_i(1,1) = 0.8[-0.04 + U_i(1,2)] + 0.1[-0.04 + U_i(2,1)] + 0.1[-0.04 + U_i(1,1)]$$
$$U_i(1,2) = 0.8[-0.04 + U_i(1,3)] + 0.2[-0.04 + U_i(1,2)]$$
$$\vdots$$

Source: RN 17.2.2

Example: Implementing Policy Improvement



Assume: $\pi_i(s)$

- Policy improvement:
  - For all the states, to find best policy:

$$\pi^*(s) = \underset{a \in A(s)}{\operatorname{argmax}} \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U(s')]$$

  - In state (1, 1):
    - Check if any action a when applied to s is better than current policy $\pi_i(s)$;
    - Update if necessary

$$\pi_{i+1}(1,1) = \underset{a \in A(s)}{\operatorname{argmax}} \sum_{s'} P(s'|(1,1),a)[R((1,1),a,s') + \gamma U(s')]$$

$$\begin{aligned}
\{\ & [0.8(-0.04+\gamma U(1,2))+0.1(-0.04+\gamma U(2,1))+0.1(-0.04+\gamma U(1,1))], && \text{(U)}\\
& [0.9(-0.04+\gamma U(1,1))+0.1(-0.04+\gamma U(1,2))], && \text{(L)}\\
& [0.9(-0.04+\gamma U(1,1))+0.1(-0.04+\gamma U(2,1))], && \text{(D)}\\
& [0.8(-0.04+\gamma U(2,1))+0.1(-0.04+\gamma U(1,2))+0.1(-0.04+\gamma U(1,1))]\} && \text{(R)}
\end{aligned}$$

Demos: Policy Iteration

IIIT Hyderabad Virtual Labs

0.000 ←	0.000 →	1.000
0.000	0.000 ←	-1.000
0.000 ←	0.000 ←	0.000 ←

https://ai1-iiith.vlabs.ac.in/exp/policy-iteration/simulation.html

Policy Iteration: Why Does It Work?

- Convergence
  - Policy evaluation and improvement converge to a fixed point.
  - Final utilities solve the Bellman equations; final policy is optimal.
  - With finite policies, termination is guaranteed.
- Optimal Policy
  - Follows the Policy Improvement Theorem [SB 4.2]:
    Each new policy is at least as good as the last.
  - Guarantees convergence to the optimal policy.

Value Iteration vs Policy Iteration

Aspect	Value Iteration	Policy Iteration
Initialization	Arbitrary utility values (e.g., $U(s) = 0$)	Arbitrary initial policy π_0
Main Update	Bellman optimality update: $U_{i+1}(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s'\|s,a)[R(s,a,s') + \gamma U_i(s')]$	1. **Policy Evaluation**: $U_i(s) = \sum_{s'} P(s'\|s,\pi_i(s))[R(s,\pi_i(s),s') + \gamma U_i(s')]$ (or use iterative updates) 2. **Policy Improvement**: $\pi_{i+1}(s) = \arg\max_a \sum_{s'} P(s'\|s,a)[R(s,a,s') + \gamma U_i(s')]$
Termination	Utility updates stop when: $\max_s \|U_{i+1}(s) - U_i(s)\| < \epsilon$	Policy improvement stops when: $\pi_{i+1}(s) = \pi_i(s)\ \forall s \in S$
Convergence	Utilities converge to a fixed point (Solution to Bellman optimality equation)	Policy converges to a stable policy (Utilities satisfy Bellman expectation equation for the current policy)
Optimality	Final policy extracted from stable utilities is guaranteed optimal	Final stable policy π^* is guaranteed optimal
Per-Iteration Cost	$O(\|S\|^2 \cdot \|A\|)$	**Policy Evaluation** (exact): $O(\|S\|^3)$ **Policy Improvement**: $O(\|S\|^2 \cdot \|A\|)$ → For each state, evaluate all actions over successor states
Remarks	Simpler update, but more iterations needed	Fewer iterations, but each iteration is more computationally intensive

Example: Persy the Mars Rover Adventures

Percy is exploring Mars:

- Starting at the base station with limited energy.
- Percy can
  - Move to explore,
  - Collect a soil sample,
  - Transmit data back to Earth, or
  - Recharge using solar panels.
- Moving and collecting are uncertain
  - Rocky terrain may drain more energy, solar recharge may fail if it's cloudy
- Goal:
  - Percy's goal is to maximize useful science data sent back before energy runs out.

Graphics: Generated by Dalle-3 Beta

Question:

- How should Percy decide what to do at each step, given uncertainty and long-term goals?

Example: Ride-Hailing Fleet Management

Real-world scenario

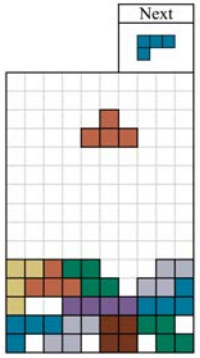
- A ride-hailing company (e.g., Grab, Uber, Lyft) manages thousands of drivers across a city:
- Each driver must decide whether to
  - Wait for a nearby passenger,
  - Relocate to a busier area, or
  - Refuel/Recharge.
- Passenger demand is uncertain, and traffic/weather affect travel times.
- Goal:
  - To maximize long-term efficiency: serve more rides, reduce idle driving, and keep drivers profitable.

Graphics generated by Dalle-3 Beta

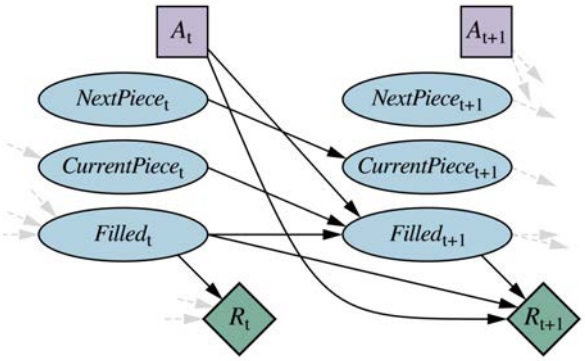
Question:

- How should the system assign and guide drivers to balance supply, demand, and future opportunities?

Fun Exercise: Can Tetris be solved using value iteration or policy iteration?



- Value Iteration:
  - Not feasible with basic algorithm
  - State space too large: 2^N for N cells
  - Iterating through all states is impractical
- Policy Iteration:
  - Similar limitations as value iteration
  - High computational cost for large state spaces



Source: RN Figure 17.5

- Approximate Methods:
  - Modified Policy Iteration (MPI): Limited iterations of simplified value iteration
  - Approximate Dynamic Programming (ADP): Uses function approximation for tractability
  - Fitted Value Iteration: Samples states to reduce computation
  - …
- Find out more about recent research!

Handling Curse of Dimensionality

When full planning or full MDP training is Infeasible

- Alternative: Reinforcement Learning (RL)
  - Learn good behavior from interaction without solving the full MDP explicitly
  - Learning without a known model
  - Approximate solutions in large-scale problems

Beyond this course:

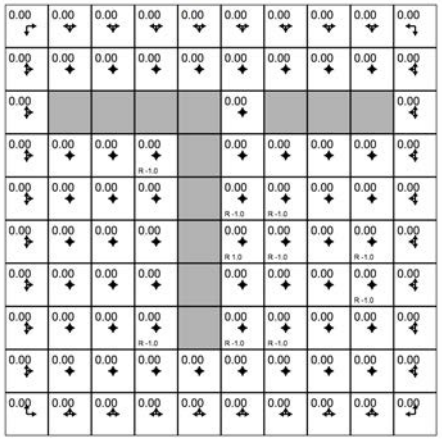
- Decision-time planning:
  - Online search at runtime
- Demonstration-based learning:
  - Learn from expert data
- Model-free RL:
  - Function approximation, policy search
- Model-based RL:
  - Learn or use world models (may include human priors)
- Large Foundation Models (LFMs)-assisted/Human-Guided Planning :
  - Model creation, task decomposition, heuristics, plan generation

Homework

- Readings
- Main topics:
  - [RN] 17.1, 17.2.1, 17.2.2
  - [SB] Chapters 3, 4 (*4.2 - Policy improvement*)
- *Online algorithms, MCTS: (Optional)*
  - *[RN] 17.2.4, 5.4*
  - *[SB] 8.10, 8.11*
- References
  - [RN] Russell, S. and P. Norvig, Artificial intelligence: A modern approach. 4th ed. 2021: Pearson. [Kindle Edition]
  - [SB] Sutton, R. S. and A. G. Barto. Reinforcement Learning: An introduction. 2nd ed. MIT Press, 2018, 2020
    [Book website: http://incompleteideas.net/book/the-book.html]
    [e-Book for personal use: http://incompleteideas.net/book/RLbook2020.pdf]

Demos: Value Iteration and Policy Iteration

Stanford REINFORCEjs Gridworld



https://cs.stanford.edu/people/karpathy/reinforcejs/gridworld_dp.html

IIIT Hyderabad Virtual Labs

https://ai1-iiith.vlabs.ac.in/List%20of%20experiments.html

Appendix

Convergence and Computing Optimal Policy of Value Iteration
Policy Improvement Theorem

Recall: Some Terminology

- Definitions:
  - A fixed point is any input unchanged by a function
  - A contraction is a function, when applied to two inputs, produces two outputs that are closer together
    - E.g., “divide by two” is a contraction
    - A contraction has only one fixed point
    - When contraction is applied to any argument, the value must get closer to the fixed point; repeated application reaches fixed point in the limit
  - Max norm or distance between two vectors is the maximum difference between any two corresponding elements
    - It measures “length” of a vector by the absolute value of its biggest component

Notes: Preference Independence Assumption (for Infinite Rewards)

- Definition
  - Each transition $s_t \xrightarrow{a_t} s_{t+1}$ is part of the history $[s_0, a_0, s_1, a_1, s_2, s_3 \ldots]$
  - Preferences between state sequences are stationary
  - If two histories $[s_0, a_0, s_1, a_1, s_2, \ldots]$and $[s'_0, a'_0, s'_1, a'_1, s'_2, \ldots]$ begin with the same transition ($s_0 = s'_0, a_0 = a'_0$, and $s_1 = s'_1$)
  - Then their preference ordering depends only on the remaining parts: $[s_1, a_1, s_2, \ldots]$and $[s'_1, a'_1, s'_2, \ldots]$
- Intuition
  - Only the future matters, not the path taken to get there.
  - If two histories start the same way, their preference depends only on what happens next.

Convergence of Value Iteration

- Value iteration:
  - Converges to the (unique) utility function for discounted problems with $\gamma < 1$
- The Bellman update $U_{i+1} \leftarrow BU_i$, is a contraction by a factor of γ on the space of utility vectors:

Bellman update operator

$$\|BU_i - BU_i'\| \leq \gamma \|U_i - U_i'\|$$

  - where max norm $\|U\| = \max_s U(s)$; $\|U - U'\| = \max_s \|U(s) - U'(s)\|$
- For Bellman equations, the utility or value function U is a fixed point

$$U = BU$$

Convergence

- Repeated application of a contraction reaches a unique fixed point U

  $\|U_{i+1} - U\| = \|BU_i - BU\| \leq \gamma\|U_i - U\| \leq \gamma^i\|U_0 - U\|$ for any initial U_0
- Reaching fixed point:
  - $U = BU$ is the fixed point utility function
  - Error $\|U_i - U\|$ reduced by a factor of at least γ on each iteration; converges exponentially fast
  - Utilities of all states bounded by $\pm R_{max}/(1-\gamma)$:

$$\|U_0 - U\| \leq 2R_{max}/(1-\gamma)$$

If U_0 is initialized to 0, then $\|U_0 - U\| \leq \frac{R_{max}}{1-\gamma}$

Convergence

- Factors influencing convergence:
  - N iterations to reach error of at most ϵ:

$$\|U_N - U\| \leq \gamma^N \cdot 2R_{max}/(1-\gamma) \leq \epsilon$$

$$N = \left\lceil \frac{\log\left(\frac{2R_{max}}{\epsilon(1-\gamma)}\right)}{\log\left(\frac{1}{\gamma}\right)} \right\rceil$$

- Termination condition:
  - If the update is small (i.e., no state utility changes by much), then the error, compared with the true utility function, also is small

$$\text{if} \quad \|U_{i+1} - U_i\| < \frac{\epsilon(1-\gamma)}{\gamma} \quad \text{then} \quad \|U_{i+1} - U\| < \epsilon$$

Convergence: Calculations

Values at each state bounded by $\pm\frac{R_{max}}{1-\gamma}$
The 2 in the numerator reflects the bounds

- If we run N iterations, we get:

Error

$$\|U_N - U\| \leq \frac{\gamma^N 2R_{max}}{1-\gamma}$$

- To get error at most ϵ, we have:

$$\frac{\gamma^N 2R_{max}}{1-\gamma} \leq \epsilon$$

$$\gamma^N 2R_{max} \leq \epsilon(1-\gamma)$$

$$\frac{2R_{max}}{\epsilon(1-\gamma)} \leq \frac{1}{\gamma^N} = \left(\frac{1}{\gamma}\right)^N$$

- Take log:

$$N \log\left(\frac{1}{\gamma}\right) \geq \log\left(\frac{2R_{max}}{\epsilon(1-\gamma)}\right)$$

$$N = \left\lceil \frac{\log\left(\frac{2R_{max}}{\epsilon(1-\gamma)}\right)}{\log\left(\frac{1}{\gamma}\right)} \right\rceil$$

- Terminating condition:
  - $\|U_{t+1} - U_t\| \leq \epsilon(1-\gamma)/\gamma$
  - $\Rightarrow \|U_{t+1} - U\| \leq \epsilon$

Computing Optimal Policy

- Question:
  - Does an MEU policy based on estimated utilities behave optimally?
- Definition:
  - Let π_i be the MEU policy wrt current estimate U_i
  - Actual utility under this policy: $U^{\pi_i}(s)$
  - Policy loss:

$$\|U^{\pi_i} - U\| = \text{max loss by following } \pi_i \text{ instead of } \pi^*.$$

- Error Bound:
  - If $\|U_i - U\| < \epsilon$ then $\|U^{\pi_i} - U\| < 2\epsilon$
- In practice:
  - Policies π_i often become optimal long before utilities U_i converge

Policy Improvement Theorem

- Let:
  - $Q^{\pi}(s,a) = \sum_{s'} P(s'|s,a)[R(s,a,s') + \gamma U^{\pi}(s')$ be the action-utility function of π (one-step look-ahead using action a)
- Theorem: [Proof in SB 4.2]
  - For two deterministic policies π and π' ,if for all $s \in S$:

$$Q^{\pi}(s,\pi'(s)) \geq U^{\pi}(s)$$

  - Then

$$U^{\pi'}(s) \geq U^{\pi}(s) \text{ for all } s \in S$$

$U(s) = \max_a Q(s,a)$

  - If the inequality is strict for any state s, then the improvement is also strict for that state.

Source: Sutton and Barto, 2020 Section 4.2

Policy Improvement theorem – Proof

- Start with $Q^{\pi}\big(s, \pi'(s)\big) \geq U^{\pi}(s)$ and keep expanding the policy

$$
\begin{aligned}
&U^{\pi}(s) \leq Q^{\pi}\big(s, \pi'(s)\big) \\
&= R(s) + \gamma \textstyle\sum_{s'} P(s'|s, \pi'(s)) U^{\pi}(s) \\
&= E_{\pi'}[R(S^t) + \gamma U^{\pi}(S^{t+1}) | S^t = s] \\
&\leq E_{\pi'}[R(S^t) + \gamma E_{\pi'}[R(S^{t+1}) + \gamma U^{\pi}(S^{t+2}) | S^{t+1}] | S^t = s] \\
&= E_{\pi'}[R(S^t) + \gamma R(S^{t+1}) + \gamma^2 U^{\pi}(S^{t+2}) | S^t = s] \\
&\leq E_{\pi'}[R(S^t) + \gamma R(S^{t+1}) + \gamma^2 R(S^{t+2}) + \gamma^3 U^{\pi}(S^{t+3}) | S^t = s] \\
&\vdots \\
&\leq E_{\pi'}[R(S^t) + \gamma R(S^{t+1}) + \gamma^2 R(S^{t+2}) + \gamma^3 R(S^{t+3}) + \cdots | S^t = s] \\
&= U^{\pi'}(s)
\end{aligned}
$$

- If the first inequality is strict, we have $U^{\pi}(s) < U^{\pi'}(s)$ $\square$

Source: Sutton and Barto, 2018/2020 Section 4.2